

Estructuras de Datos

Clase 16 – Ejercicios de árboles



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



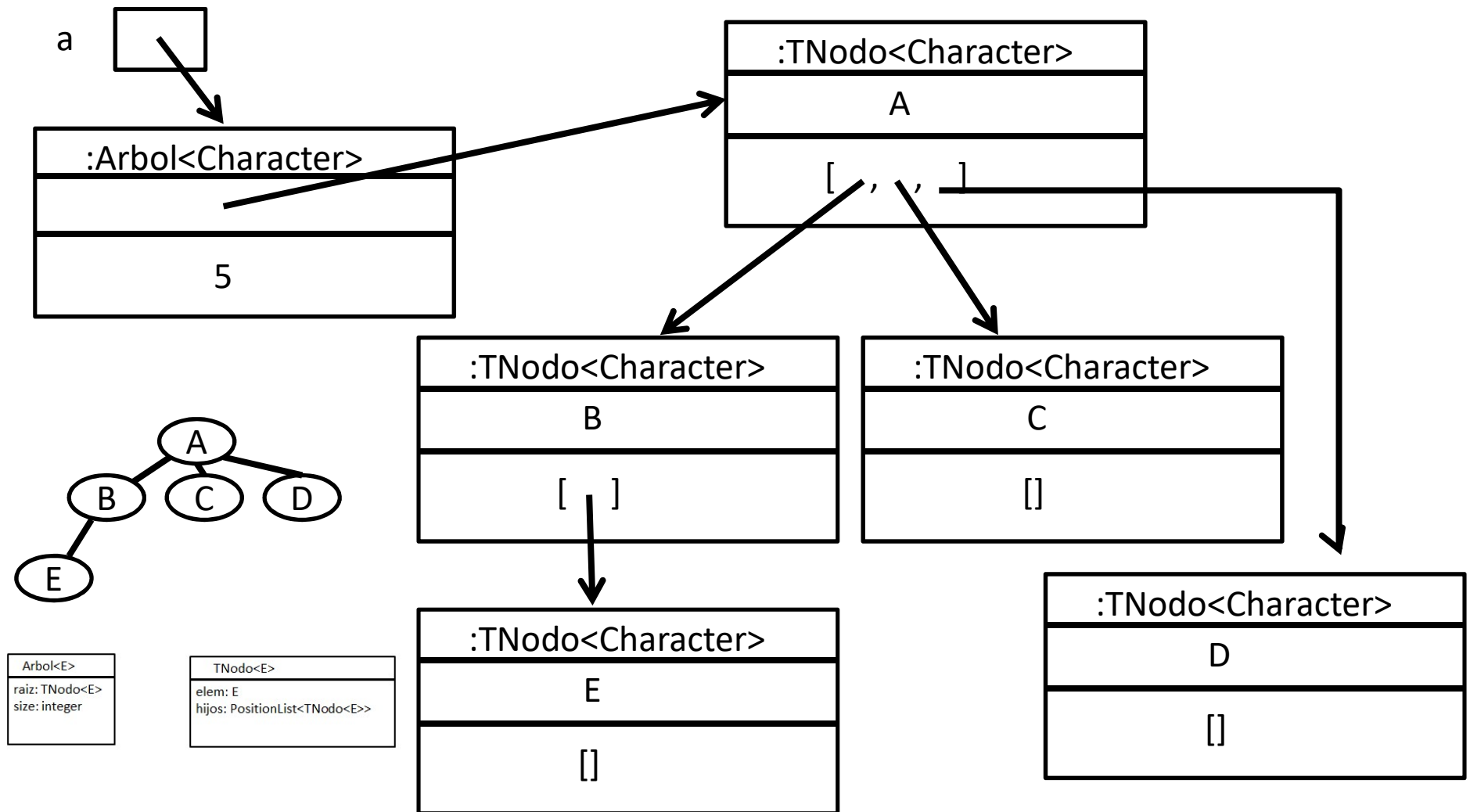
Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

```

public interface Tree<E> extends Iterable<E>
{
    public int size();
    public boolean isEmpty();
    public Iterator<E> iterator();
    public Iterable<Position<E>> positions();
    public E replace(Position<E> v, E e); // throws InvalidPositionException;
    public Position<E> root() ; // throws EmptyTreeException;
    public Position<E> parent(Position<E> v) ;
        // throws InvalidPositionException, BoundaryViolationException;
    public Iterable<Position<E>> children(Position<E> v) ; // throws InvalidPositionException;
    public boolean isInternal(Position<E> v) ; // throws InvalidPositionException;
    public boolean isExternal(Position<E> v) ; // throws InvalidPositionException;
    public boolean isRoot(Position<E> v) ; // throws InvalidPositionException;
    public void createRoot(E e) ; // throws InvalidOperationException;
    public Position<E> addFirstChild(Position<E> p, E e) ; // throws InvalidPositionException;
    public Position<E> addLastChild(Position<E> p, E e) ; // throws InvalidPositionException;
    public Position<E> addBefore(Position<E> p, Position<E> rb, E e) ;
        // throws InvalidPositionException;
    public Position<E> addAfter (Position<E> p, Position<E> lb, E e) ;
        // throws InvalidPositionException;
    public void removeExternalNode (Position<E> p) ; // throws InvalidPositionException;
    public void removeInternalNode (Position<E> p) ; // throws InvalidPositionException;
    public void removeNode (Position<E> p) ; // throws InvalidPositionException;
}

```

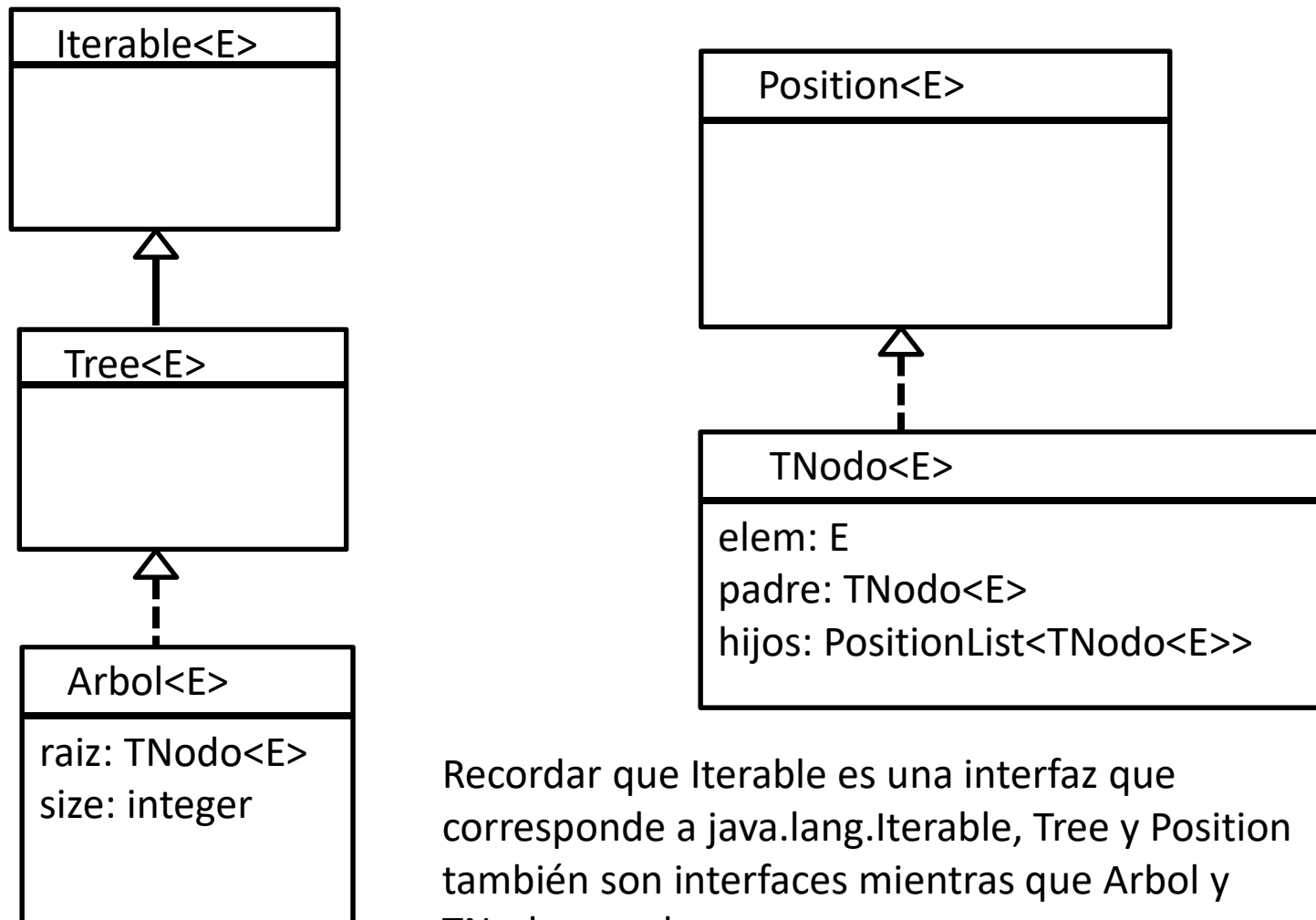
Representación de lista de hijos



Representación en GT: Colección de hijos

- GT = Lista de hijos + padre
- El árbol conoce la raíz
- Cada nodo conoce el rótulo (elemento), la lista de nodos hijos y el nodo padre.

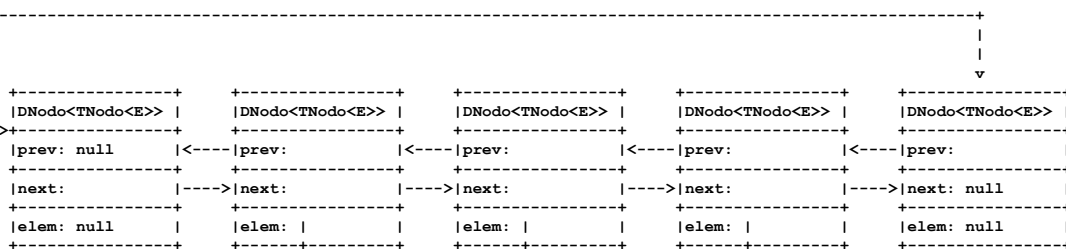
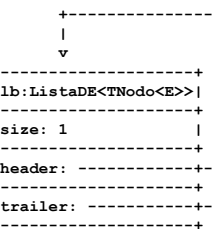
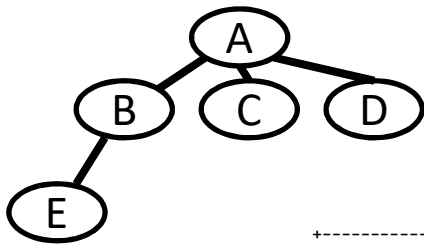
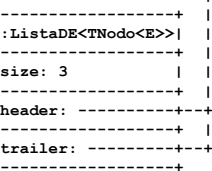
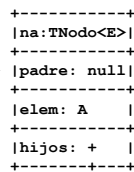
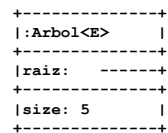
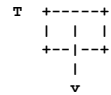
Representación de árboles en GT



Recordar que Iterable es una interfaz que corresponde a java.lang.Iterable, Tree y Position también son interfaces mientras que Arbol y TNode son clases.

Representación de árboles en GT: Diagrama de objetos

Tree<E> T



Abuso de notación:

A las listas de hijos de C, D, E simplemente las graficamos como [] porque están vacías.

Problema 1

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase. Esta clase utiliza la implementación de lista de hijos y enlace al padre. Agregue un método a esta clase cuya signatura sea **public int sizeSubarbol(Position<E> p) throws InvalidPositionException**. Este método deberá retornar el tamaño del subárbol con raíz p del árbol receptor del mensaje. Si utiliza otros métodos del TDA Arbol deberá implementarlos.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Problema 2

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase. Esta clase utiliza la implementación de lista de hijos y enlace al padre. Agregue un método a esta clase cuya signatura sea **public Map<Position<E>, Integer> mapSizeSubarboles() throws InvalidPositionException**. Este método deberá retornar un mapeo entre posiciones del árbol y el tamaño del subárbol que tiene a esa posición como raíz. Si utiliza otros métodos del TDA Arbol deberá implementarlos.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Problema 3

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase. Esta clase utiliza la implementación de lista de hijos y enlace al padre. Agregue un método a esta clase cuya signatura sea **public int podarSubarbol(Position<E> p) throws InvalidPositionException**. Este método deberá eliminar el subarbol con raíz p del árbol receptor del mensaje. Retorna el tamaño del subárbol eliminado. Si utiliza otros métodos del TDA Arbol deberá implementarlos.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Problema 4

Suponiendo que cuenta con una clase *Arbol* $\langle E \rangle$ que implementa la interfaz *Tree* $\langle E \rangle$ vista en clase y trabajando en función de las operaciones del TDA Árbol, resuelva el siguiente problema; dado un árbol T y dos rótulos e y f , se le debe cambiar el rótulo a todos los nodos del árbol que tengan el rótulo e y por f .

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Problema 5

Utilizando el TDA Árbol, resuelva el siguiente problema; dado un árbol A, un rótulo e y un nivel x , se le debe insertar un nuevo primer hijo con rótulo e a cada nodo en el nivel x . El método a implementar debe tener la siguiente forma:

```
public static void insertarMasivo( E e, int x, Tree<E> A )
```

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

